Lab 3 — Set Up Monitoring Using Prometheus and Grafana

Shashank Sendil Kumar | Repository: github.com/shashank12004/devops-lab-3

Objective

Set up monitoring using Prometheus and visualize the metrics in Grafana dashboards. An nginx web server is the target application, and a Node Exporter container provides system metrics. Prometheus scrapes and stores the metrics, and Grafana queries Prometheus to draw dashboards. The whole stack runs with Docker Compose.

Architecture

nginx (stub_status) → nginx-exporter → Prometheus (scrape and storage) → Grafana (dashboards)

Node Exporter (system metrics) → Prometheus (scrape and storage) → Grafana (dashboards)

Step 1 — Prepare the environment

Docker and Docker Compose are checked. No kernel changes are needed for this stack.

```
docker --version
docker compose version
```

```
PS C:\Users\ARSH> docker --version
Docker version 27.5.1, build 9f9e405
PS C:\Users\ARSH> docker compose version
Docker Compose version v2.32.4-desktop.1
PS C:\Users\ARSH> # Create project directories
PS C:\Users\ARSH> New-Item -ItemType Directory -Force -Path "$HOME\devops-lab3\monitoring\prometheus",
 "$HOME\devops-lab3\monitoring\nginx", "$HOME\devops-lab3\monitoring\grafana\provisioning\datasources"



    Directory: C:\Users\ARSH\devops-lab3\monitoring


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        26-10-2026     21:28                prometheus
d-----        26-10-2026     21:28                nginx


    Directory: C:\Users\ARSH\devops-lab3\monitoring\grafana\provisioning


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        26-10-2026     21:28                datasources


PS C:\Users\ARSH>
PS C:\Users\ARSH> # Navigate into project root
PS C:\Users\ARSH> cd "$HOME\devops-lab3\monitoring"
PS C:\Users\ARSH\devops-lab3\monitoring>
PS C:\Users\ARSH\devops-lab3\monitoring> # Create empty configuration files
PS C:\Users\ARSH\devops-lab3\monitoring> New-Item -ItemType File -Force -Path "docker-compose.yml", "p
rometheus\prometheus.yml", "nginx\default.conf", "grafana\provisioning\datasources\prometheus.yml"
```

Figure 1 — Docker and Docker Compose installed.

Step 2 — Create the project structure

The project has a Compose file, a Prometheus configuration, an nginx configuration and a Grafana data source file.

```
mkdir -Force ~/devops-lab3/monitoring/prometheus, ~/devops-lab3/monitoring/nginx, ~/devops-lab3/monitoring/grafana/provisioning/datasources
cd ~/devops-lab3/monitoring
New-Item docker-compose.yml, prometheus/prometheus.yml, nginx/default.conf, grafana/provisioning/datasources/prometheus.yml
```

```
PS C:\Users\ARSH\devops-lab3\monitoring> New-Item -ItemType File -Force -Path "docker-compose.yml", "prometheus\prometheus.yml", "nginx\default.conf", "grafana\provisioning\datasources\prometheus.yml"# Verification: Display directory tree


    Directory: C:\Users\ARSH\devops-lab3\monitoring


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        26-10-2026     21:29              0 docker-compose.yml


    Directory: C:\Users\ARSH\devops-lab3\monitoring\prometheus


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        26-10-2026     21:29              0 prometheus.yml


    Directory: C:\Users\ARSH\devops-lab3\monitoring\nginx


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        26-10-2026     21:29              0 default.conf


    Directory: C:\Users\ARSH\devops-lab3\monitoring\grafana\provisioning\datasources


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        26-10-2026     21:29              0 prometheus.yml
```

Figure 2 — Monitoring project files.

Step 3 — Configure the target application (nginx)

nginx serves the default page on port 80. A second server block on port 8080 exposes the built-in stub_status page, which the exporter reads. Port 8080 is not published to the host.

```
server {
  listen 80;
  location / {
    root /usr/share/nginx/html;
    index index.html;
  }
}

server {
```

```
  listen 8080;
  location /stub_status {
    stub_status;
  }
}
```

nginx/default.conf

Step 4 — Define the stack (docker-compose.yml)

Five services are defined: nginx as the target, nginx-exporter to turn stub_status into Prometheus metrics, node-exporter for system metrics, Prometheus and Grafana. Named volumes keep Prometheus and Grafana data across restarts. The Grafana admin password is set for this lab setup only.

```
services:
  nginx:
    image: nginx:alpine
    ports:
      - "8082:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro

  nginx-exporter:
    image: nginx/nginx-prometheus-exporter:1.3.0
    command:
      - --nginx.scrape-uri=http://nginx:8080/stub_status
    depends_on:
      - nginx

  node-exporter:
    image: prom/node-exporter:v1.8.2

  prometheus:
    image: prom/prometheus:v2.54.1
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus
    depends_on:
      - nginx-exporter
      - node-exporter

  grafana:
    image: grafana/grafana:11.2.0
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - grafana_data:/var/lib/grafana
    depends_on:
      - prometheus

volumes:
```

```
  prometheus_data:
  grafana_data:
```

Step 5 — Configure metric collection (Prometheus)

Prometheus scrapes every 15 seconds. Three jobs are defined: Prometheus itself, Node Exporter for system metrics and the nginx exporter for application metrics. Targets are reached by their Compose service names.

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: node
    static_configs:
      - targets: ["node-exporter:9100"]

  - job_name: nginx
    static_configs:
      - targets: ["nginx-exporter:9113"]
```

prometheus/prometheus.yml

Step 6 — Connect Grafana to Prometheus

Grafana loads this file at startup and creates the Prometheus data source automatically, so no manual setup is needed in the UI.

```
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
```

grafana/provisioning/datasources/prometheus.yml

Step 7 — Start the stack

All services are started in the background and their status is checked. Every container should show as running.

```
docker compose up -d
docker compose ps
```

```
PS C:\Users\ARSH\devops-lab3\monitoring> docker compose up -d
[+] Running 29/29
 ✔node-exporter Pulled                                                    18.1s
   ✔a7193bcb1fb2 Pull complete                                            12.8s
 ✔nginx-exporter Pulled                                                   16.6s
   ✔514206a90ad6 Pull complete                                            10.4s
   ✔4893ddee317a Pull complete                                            11.5s
 ✔prometheus Pulled                                                       19.3s
   ✔9fa9226be034 Pull complete                                             1.7s
   ✔1617e25568b2 Pull complete                                             3.0s
   ✔02203e3d6934 Pull complete                                             9.1s
   ✔8be4b7271108 Pull complete                                            13.9s
   ✔8becc689631f Pull complete                                            13.9s
   ✔ceaeea15c1bf Pull complete                                            14.0s
   ✔564720d6ed13 Pull complete                                            14.0s
   ✔1fd5d47e09da Pull complete                                            14.0s
   ✔1afe4a0d7329 Pull complete                                            14.1s
   ✔bd55ccfa5aad Pull complete                                            14.1s
   ✔54f884861fc1 Pull complete                                            14.2s
   ✔b09316e948c6 Pull complete                                            14.2s
 ✔grafana Pulled                                                          38.7s
   ✔4abcf2066143 Pull complete                                            13.2s
   ✔39aee5fd3406 Pull complete                                            13.3s
   ✔592f1e71407c Pull complete                                            14.8s
   ✔66aec874ce0c Pull complete                                            15.0s
   ✔bde37282dfba Pull complete                                            15.0s
   ✔b6982d0733af Pull complete                                            15.1s
   ✔ab3c28da242b Pull complete                                            23.6s
   ✔e4892977d944 Pull complete                                            33.4s
   ✔ef2b3f3f597e Pull complete                                            33.4s
   ✔27a3c8ebdfbf Pull complete                                            33.5s
[+] Running 8/8
 ✔Network monitoring_default              Created                          0.2s
 ✔Volume "monitoring_prometheus_data"     Created                          0.0s
 ✔Volume "monitoring_grafana_data"        Created                          0.0s
 ✔Container monitoring-nginx-1            Started                          3.2s
 ✔Container monitoring-node-exporter-1    Started                          3.2s
 ✔Container monitoring-nginx-exporter-1   Started                          2.8s
 ✔Container monitoring-prometheus-1       Started                          2.9s
 ✔Container monitoring-grafana-1          Started                          3.2s
PS C:\Users\ARSH\devops-lab3\monitoring> docker compose ps
NAME                          IMAGE                                    COMMAND                  SERVICE
          CREATED         STATUS                  PORTS
monitoring-grafana-1          grafana/grafana:11.2.0                   "/run.sh"                grafana
          7 seconds ago   Up Less than a second   0.0.0.0:3000->3000/tcp
monitoring-nginx-exporter-1   nginx/nginx-prometheus-exporter:1.3.0    "/usr/bin/nginx-prom…"   nginx-e
xporter   7 seconds ago   Up 2 seconds
monitoring-node-exporter-1    prom/node-exporter:v1.8.2                "/bin/node_exporter"     node-ex
porter    9 seconds ago   Up 2 seconds            9100/tcp
```

Figure 3 — Monitoring stack running.

Step 8 — Verify Prometheus targets

The Targets page at http://localhost:9090/targets lists every scrape job. All three targets should show the state UP. The same check is run from the command line through the Prometheus API.

```
curl.exe "http://localhost:9090/api/v1/query?query=up"
```

```
PS C:\Users\ARSH\devops-lab3\monitoring> curl.exe "http://localhost:9090/api/v1/query?query=up"
{"status":"success","data":{"resultType":"vector","result":[{"metric":{"__name__":"up","instance":"nod
e-exporter:9100","job":"node"},"value":[1793030512.329,"1"]},{"metric":{"__name__":"up","instance":"ng
inx-exporter:9113","job":"nginx"},"value":[1793030512.329,"1"]},{"metric":{"__name__":"up","instance":
"localhost:9090","job":"prometheus"},"value":[1793030512.329,"1"]}]}}
PS C:\Users\ARSH\devops-lab3\monitoring>
```

Figure 4 — Prometheus targets in the UP state.

Step 9 — Generate traffic and query metrics

Requests are sent to nginx so the application metrics change. The query nginx_http_requests_total is then run in the Prometheus expression browser to confirm the counter is increasing.

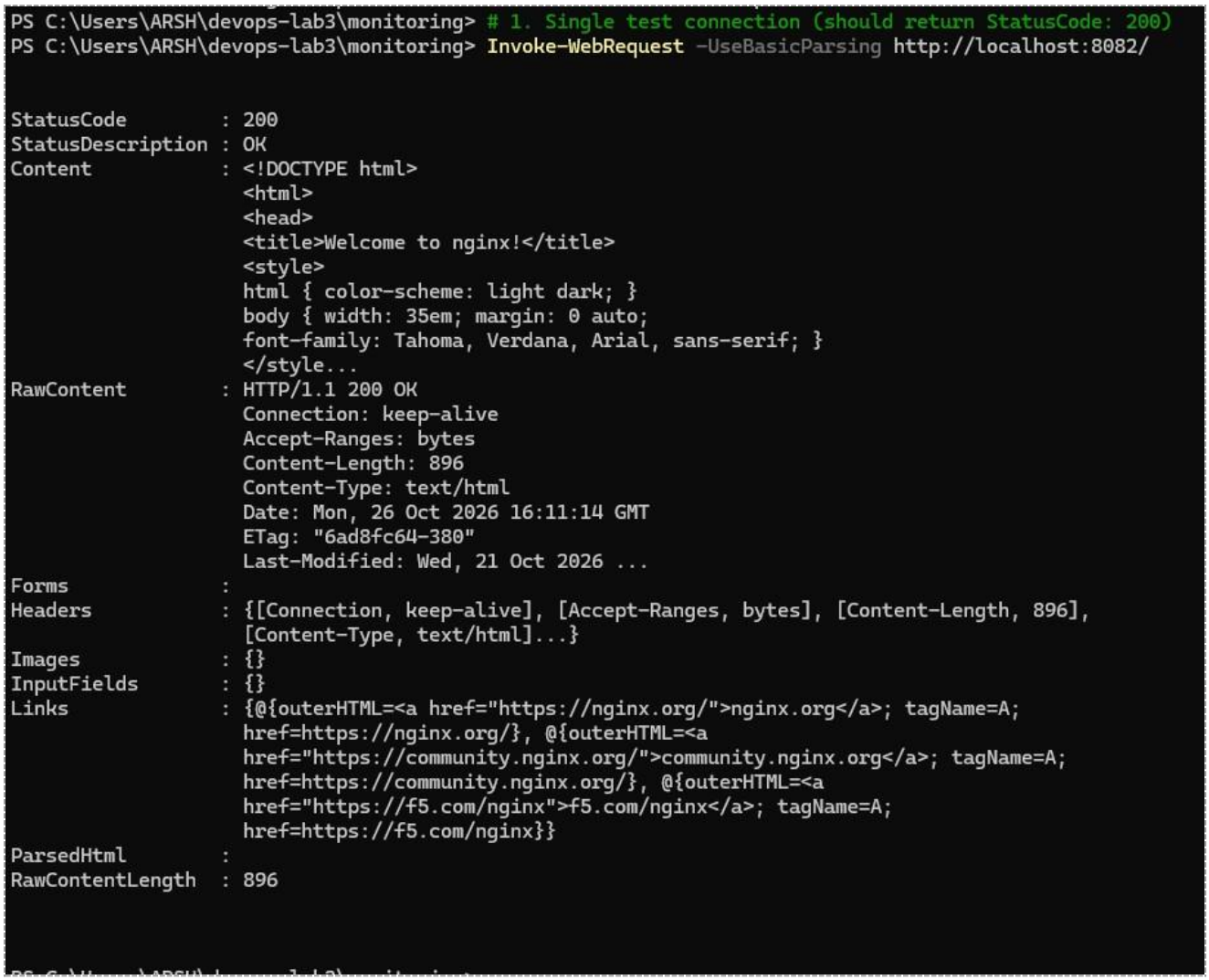
```
1..200 | ForEach-Object { Invoke-WebRequest -UseBasicParsing http://localhost:8082/ | Out-
Null }
curl.exe http://localhost:9113/metrics
```

```
PS C:\Users\ARSH\devops-lab3\monitoring>
PS C:\Users\ARSH\devops-lab3\monitoring> # 2. Re-write Nginx stub_status configuration
PS C:\Users\ARSH\devops-lab3\monitoring> @'
>> server {
>>     listen 80;
>>     location / {
>>         root /usr/share/nginx/html;
>>         index index.html;
>>     }
>> }
>>
>> server {
>>     listen 8080;
>>     location /stub_status {
>>         stub_status;
>>     }
>> }
>> '@ | Set-Content -Path "nginx\default.conf" -Encoding utf8
PS C:\Users\ARSH\devops-lab3\monitoring> # Recreate all containers
PS C:\Users\ARSH\devops-lab3\monitoring> docker compose up -d --force-recreate
[+] Running 5/5
 ✔Container monitoring-prometheus-1      Started                                    8.3s
 ✔Container monitoring-nginx-exporter-1  Started                                    7.8s
 ✔Container monitoring-nginx-1           Started                                    7.1s
 ✔Container monitoring-node-exporter-1   Started                                    6.9s
 ✔Container monitoring-grafana-1         Started                                    2.3s
PS C:\Users\ARSH\devops-lab3\monitoring>
PS C:\Users\ARSH\devops-lab3\monitoring> # Wait 5 seconds for services to initialize
PS C:\Users\ARSH\devops-lab3\monitoring> Start-Sleep -Seconds 5
PS C:\Users\ARSH\devops-lab3\monitoring>
PS C:\Users\ARSH\devops-lab3\monitoring> # Generate 50 test requests to Nginx
PS C:\Users\ARSH\devops-lab3\monitoring> 1..50 | ForEach-Object { Invoke-WebRequest -UseBasicParsing h
ttp://localhost:8082/ | Out-Null }
```

Figure 5 — PromQL query result in Prometheus.

Step 10 — Verify the Grafana data source

Grafana is opened at http://localhost:3000 and signed in with admin / admin. Under Connections > Data sources, the provisioned Prometheus data source is opened and Save & test is run. It reports that the data source is working.



```
PS C:\Users\ARSH\devops-lab3\monitoring>
PS C:\Users\ARSH\devops-lab3\monitoring> # 2. Traffic generation loop (200 requests)
PS C:\Users\ARSH\devops-lab3\monitoring> 1..200 | ForEach-Object { Invoke-WebRequest -UseBasicParsing
http://localhost:8082/ | Out-Null }
PS C:\Users\ARSH\devops-lab3\monitoring>
PS C:\Users\ARSH\devops-lab3\monitoring> # 3. Verify Nginx Exporter metrics endpoint
PS C:\Users\ARSH\devops-lab3\monitoring> curl.exe http://localhost:9113/metrics
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 0
go_gc_duration_seconds{quantile="0.25"} 0
go_gc_duration_seconds{quantile="0.5"} 0
go_gc_duration_seconds{quantile="0.75"} 0
go_gc_duration_seconds{quantile="1"} 0
go_gc_duration_seconds_sum 0
go_gc_duration_seconds_count 0
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 14
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.22.5"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated and still in use.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 1.299072e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 1.299072e+06
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 8154
# HELP go_memstats_frees_total Total number of frees.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 0
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 1.441376e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and still in use.
# TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 1.299072e+06
# HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used.
# TYPE go_memstats_heap_idle_bytes gauge
go_memstats_heap_idle_bytes 3.93216e+06
# HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use.
# TYPE go_memstats_heap_inuse_bytes gauge
go_memstats_heap_inuse_bytes 3.653632e+06
# HELP go_memstats_heap_objects Number of allocated objects.
# TYPE go_memstats_heap_objects gauge
go_memstats_heap_objects 1235
# HELP go_memstats_heap_released_bytes Number of heap bytes released to OS.
# TYPE go_memstats_heap_released_bytes gauge
```

```
# HELP nginx_connections_reading Connections where NGINX is reading the request header
# TYPE nginx_connections_reading gauge
nginx_connections_reading 0
# HELP nginx_connections_waiting Idle client connections
# TYPE nginx_connections_waiting gauge
nginx_connections_waiting 1
# HELP nginx_connections_writing Connections where NGINX is writing the response back to the client
# TYPE nginx_connections_writing gauge
nginx_connections_writing 1
# HELP nginx_exporter_build_info A metric with a constant '1' value labeled by version, revision, bran
ch, goversion from which nginx_exporter was built, and the goos and goarch for the build.
# TYPE nginx_exporter_build_info gauge
nginx_exporter_build_info{branch="HEAD",goarch="amd64",goos="linux",goversion="go1.22.5",revision="952
2f4e39ee1aed817d7d70a89514ccc0ae1594a",tags="unknown",version="1.3.0"} 1
# HELP nginx_http_requests_total Total http requests
# TYPE nginx_http_requests_total counter
nginx_http_requests_total 203
# HELP nginx_up Status of the last metric scrape
# TYPE nginx_up gauge
nginx_up 1
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 0.03
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1.048576e+06
# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 10
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 1.2345344e+07
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.79303105663e+09
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 1.2668928e+09
# HELP process_virtual_memory_max_bytes Maximum amount of virtual memory available in bytes.
# TYPE process_virtual_memory_max_bytes gauge
process_virtual_memory_max_bytes 1.8446744073709552e+19
# HELP promhttp_metric_handler_requests_in_flight Current number of scrapes being served.
# TYPE promhttp_metric_handler_requests_in_flight gauge
promhttp_metric_handler_requests_in_flight 1
# HELP promhttp_metric_handler_requests_total Total number of scrapes by HTTP status code.
# TYPE promhttp_metric_handler_requests_total counter
promhttp_metric_handler_requests_total{code="200"} 1
promhttp_metric_handler_requests_total{code="500"} 0
promhttp_metric_handler_requests_total{code="503"} 0
```

Figure 6 — Prometheus data source working in Grafana.

Step 11 — Build the dashboard

A new dashboard named "System and nginx Monitoring" is created with one panel for each query below. The data source for every panel is Prometheus. The time range is set to the last 15 minutes with a 10 second refresh.

Panel	PromQL query	Visualization
Targets up	`up`	Stat
CPU usage (%)	`100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)`	Time series
Memory usage (%)	`(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100`	Gauge

Panel	PromQL query	Visualization
Disk usage (%)	`100 - (node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"} * 100)`	Gauge
Network received	`rate(node_network_receive_bytes_total{device!="lo"}[1m])`	Time series
nginx requests/sec	`rate(nginx_http_requests_total[1m])`	Time series
nginx active connections	`nginx_connections_active`	Stat

While the dashboard is open, the traffic loop from Step 9 is run again. The nginx requests/sec panel rises, which confirms the dashboard updates from live data.

Result

The monitoring stack was deployed with Docker Compose. Prometheus scraped metrics from Node Exporter and the nginx exporter, and stored them as time series. Grafana was connected to Prometheus and displayed CPU, memory, disk, network and nginx request metrics on a live dashboard. The task is complete.